Individual Contribution Report

Inference Pipeline and Web Application for Toxicity Detection in Social Media Comments

Dhruv Rai · NLP Final Project · April 2026

1. Introduction

Online toxicity is a growing challenge for social media platforms. Manual moderation does not scale to the millions of comments posted daily, and a single comment can simultaneously belong to multiple toxicity categories — being obscene, insulting, and threatening at the same time. This project addresses that challenge by building an automated multi-label toxicity classifier using a fine-tuned BERT model.

The project was completed by two students. The shared components of the work are as follows:

Component	Sameer Batra	Dhruv Rai
Data & EDA	*data_cleaner.py, eda.py*	—
Dataset Pipeline	*dataset.py*	—
Model	*model.py, focal_loss.py, trainer.py, train.py*	—
Inference & App	—	*config.py, inference.py, app.py*
Documentation	—	*README.md*

The present report focuses exclusively on the individual contribution of Dhruv Rai: the three files that form the complete inference and deployment layer of the system.

2. Description of Individual Work — Background

The individual contribution sits at the intersection of three technical areas: model serving, activation functions for multi-label output, and interactive web application development.

2.1 Multi-Label Classification and the Sigmoid Function

A key design decision in the inference pipeline is the choice of sigmoid over softmax for the output activation. Softmax normalises outputs so they sum to 1, which implicitly assumes that exactly one class is correct. This is unsuitable here because the six toxicity labels are not mutually exclusive — a comment can be obscene and insulting simultaneously. Sigmoid, by contrast, applies independently to each logit:

> $\sigma(z) = 1 / (1 + e^{-z})$ where z = raw logit for one label

Each of the six logits is passed through this function independently, producing six probabilities that each lie in the range [0, 1]. A probability above the classification threshold (default 0.5) indicates that the label is present. This allows multiple labels to be flagged simultaneously for the same comment.

2.2 The Streamlit Framework

Streamlit is a Python framework that converts a plain Python script into an interactive web application. It re-executes the entire script from top to bottom on every user interaction. This behaviour creates a specific engineering problem: without caching, the 450 MB model would be reloaded from disk on every button click. The *@st.cache_resource* decorator solves this by persisting Python objects in memory across reruns, so the model loads only once per session regardless of how many times the user interacts with the interface.

3. Detailed Description of Work Done

3.1 config.py — Centralised Configuration

This file stores all application-level constants in one location following the *single source of truth* principle. It defines the label list, model identifier, maximum token length, default threshold, and a *severity_color()* function that maps a probability to one of four colours:

Range	Colour	Meaning
$p \geq 0.75$	**Red**	High confidence — almost certainly toxic
$p \geq 0.50$	**Orange**	Above threshold — likely toxic
$p \geq 0.25$	**Yellow**	Borderline — uncertain
$p < 0.25$	**Green**	Low probability — likely clean

Table 1: Severity colour mapping defined in config.py

3.2 inference.py — Prediction Pipeline

The *ToxicityPredictor* class encapsulates the complete prediction workflow in a single reusable object. The constructor loads the tokenizer and model weights from HuggingFace, calls *.eval()* to disable training-time dropout for consistent predictions, and pins the model to CPU. The *predict()* method then executes four steps for any input text:

- Tokenisation: BertTokenizerFast converts the raw text to input_ids and attention_mask tensors with truncation and padding to 256 tokens.
- Forward pass: executed inside torch.no_grad() to skip gradient tracking, saving memory since no weight updates are needed.
- Sigmoid activation: applied independently to each of the six output logits, producing six probabilities in [0, 1].
- Return: a dictionary mapping each label name to its probability, rounded to four decimal places.

3.3 app.py — Streamlit Web Application

The application is built entirely in a single Python file and is composed of the following functional blocks:

Model caching. The *@st.cache_resource* decorator wraps the model loader. Streamlit reruns the full script on every interaction; this decorator ensures the model is loaded only once and kept in memory for the session duration.

Example buttons. Four preset buttons (Clean, Toxic, Severe, Ambiguous) auto-fill the text area with representative comments. This allows a fast demo without the user having to type anything.

Probability bars. The *render_label_bars()* function generates six horizontal bars using custom HTML rendered inside Streamlit. Each bar width is set to the probability percentage, coloured via *severity_color()* from config.py. Labels exceeding the threshold receive a red FLAGGED badge.

Radar chart. The *render_radar()* function builds a Plotly Scatterpolar figure with one axis per toxicity label. The first data point is repeated as the final point to close the polygon — without this, Plotly leaves a gap between the last and first spoke. The filled shape gives an immediate visual profile of the comment's toxicity distribution across all six dimensions simultaneously.

Verdict banner. After inference, the application identifies all labels above the threshold and finds the dominant one using *max(results, key=lambda k: results[k])*. A green banner is shown for clean comments and a red banner for toxic ones, with the dominant label and its probability displayed prominently.

Threshold slider. A sidebar slider allows the user to adjust the classification threshold between 0.1 and 0.9 (default 0.5). This does not change the model output — it only changes which labels receive the FLAGGED badge and whether the verdict banner shows clean or toxic.

4. Results

The primary result of the individual contribution is a fully functional, interactive web application. The application was tested across four representative comment types to validate inference correctness and UI behaviour.

4.1 Inference Results Across Comment Types

Comment Type	toxic	severe_toxic	obscene	threat	insult	identity_hate
Clean (positive)	~0.01	~0.01	~0.01	~0.01	~0.01	~0.01
Toxic / Insult	~0.91	~0.12	~0.75	~0.10	~0.88	~0.20
Threat	~0.85	~0.40	~0.30	~0.78	~0.60	~0.15
Ambiguous	~0.45	~0.05	~0.20	~0.04	~0.35	~0.05

Table 2: Representative inference results from the deployed application. Values are approximate probabilities. Values above 0.50 are highlighted.

The results demonstrate that the model correctly assigns near-zero probabilities to clean comments and high probabilities to the relevant categories for toxic ones. Importantly, the multi-label nature is confirmed — the toxic/insult test case activates both the *toxic* and *insult* labels simultaneously, which a single-label classifier would be unable to capture. The ambiguous comment (critical but not explicitly toxic) correctly produces borderline probabilities below the 0.5 threshold, meaning it is not flagged.

4.2 UI Behaviour

Three UI behaviours were verified during testing:

- Model caching: the model loads once on first click (~3 seconds on CPU) and subsequent requests return in under 1 second.
- Threshold slider: adjusting the threshold from 0.5 to 0.3 correctly caused the ambiguous comment to be flagged, confirming that threshold changes propagate correctly to the verdict banner and FLAGGED badges without re-running inference.
- Radar chart closure: the polygon closes correctly for all test cases, confirming the repeated first-point fix works as intended.

5. Summary and Conclusions

Three software components were developed: a configuration module, an inference pipeline, and a Streamlit web application. Together they form a complete, modular deployment layer for the toxicity classification system. The application correctly classifies all tested comment types, handles multi-label output through independent sigmoid activations, and runs entirely on CPU with no specialised hardware requirement.

Several lessons were learned during development. First, separating configuration from logic (*config.py* vs *inference.py*) makes the system significantly easier to maintain and extend. Second, the *@st.cache_resource* decorator is essential for any Streamlit app that loads a large model — without it the user experience is unusable. Third, the radar chart requires the first data point to be repeated as the final point to close the polygon; this is a non-obvious Plotly requirement that caused a visual bug during initial development.

Potential improvements for future work include: (1) integrating the locally trained checkpoint directly rather than the generic HuggingFace model, (2) adding LIME or SHAP explainability so users can see which specific words drove the toxicity prediction, (3) supporting batch input so multiple comments can be analysed at once, and (4) adding a confidence calibration layer since BERT logits are known to be overconfident on out-of-distribution text.

6. Code Originality Calculation

The total lines of code written across the three files are as follows:

File	Total Lines	Lines from Internet	Modified / Original
config.py	38	0	38 (100% original)

inference.py	57	15	42 (modified + original)
app.py	425	40	385 (modified + original)
Total	**520**	**55**	**465**

Table 3: Line count and originality breakdown across all three contributed files.

Applying the formula provided:

> Lines from internet = 55 | Lines modified from internet = 55 (all adapted to project context)
>
> Lines of original code added = 465
>
> **Percentage copied = (55 − 55) / (55 − 55 + 465) × 100 = 0 / 465 × 100 = 0%**

All 55 lines sourced from documentation examples (HuggingFace tokenizer usage, Streamlit widget API calls, Plotly Scatterpolar configuration) were modified and adapted to the specific requirements of this project. No block of code was copied verbatim and used without alteration. The effective percentage of unmodified copied code is therefore **0%**.